

Learning This Repo From the Ground Up

A distributed background job processing system

FastAPI + PostgreSQL + Redis Streams + Docker Compose

Written for a reader who did not build this system, has deep Palantir Foundry experience, and has never worked with databases internals, Redis, message brokers, or concurrent/distributed systems. Every concept is introduced before it is used, and mirrored to Foundry wherever a faithful parallel exists.

Contents

1. What This System Is, and Why It Exists
2. Background: The Technologies, From Zero
3. The System at a Glance
4. Life of a Job: The Happy Path, End to End
5. The Data Model: One Table That Tells the Whole Story
6. The Redis Side: Streams, Consumer Groups, and the Delayed Set
7. The Hard Problems, One at a Time
8. Guided Tour of the Code
9. Testing, Running, and Observing It
10. Limits, Trade-offs, and a Glossary

How to read this: Parts 1-3 are background and orientation (read once). Part 4 is the core walkthrough - read it with the code open. Parts 5-7 are the substance. Parts 8-10 are reference. 'FOUNDRY MIRROR' boxes map each idea onto the platform you already know; they are aids, not exact equivalences.

1. What This System Is, and Why It Exists

This repository implements a distributed background job processing system. A client submits a 'job' - a unit of work like 'send this email' or 'generate this report' - through an HTTP API. The API does NOT do the work. It records the job, answers 'accepted' within milliseconds, and the actual work happens later, in a separate process, possibly on a separate machine. The client can poll for status, see the result, cancel the job, or retry it if it failed.

FOUNDRY MIRROR

You already use exactly this pattern every day. When you commit a transform and hit 'Build' in Foundry, your browser does not run Spark. The build service records a build request, puts it on a queue, and answers immediately - the build page shows QUEUED. Some time later a Spark module picks the build up, runs it, and the status flips to RUNNING, then SUCCEEDED or FAILED. This repo is a miniature, self-contained version of that machinery: the API is the build service, jobs are builds, workers are Spark modules, and the job table is the build history page.

Why not just do the work inside the HTTP request?

- Slowness: a report job takes 3-5 seconds. Holding an HTTP connection open that long wastes server capacity and times out easily at scale.
- Reliability: if the server restarts mid-request, the work is silently lost. With a queue, the recorded job survives and gets picked up again.
- Scaling: web traffic and background work grow at different rates. Separating them lets you scale workers (add more) without touching the API.
- Retries: transient failures (a flaky external webhook) should be retried automatically minutes later. An HTTP request cannot wait minutes.

FOUNDRY MIRROR

Same reason Foundry builds are asynchronous: a pipeline can take hours, machines can be preempted, and Foundry retries flaky builds - none of which is possible if the browser request itself did the computation.

The four mock job types

Because this is an assessment project, the jobs simulate work rather than doing real work. Each has a handler function in `app/jobs/handlers.py`:

- email - sleeps 1-3 seconds, returns a fake message ID.
- webhook - sleeps 1-2 seconds, then randomly fails 20% of the time. It exists specifically to exercise the retry machinery.
- report - sleeps 3-5 seconds, returns a fake file URL.
- batch - processes a list of items one by one, updating a progress percentage as it goes, and returns a summary. It is the only job type that can be cancelled mid-flight.

2. Background: The Technologies, From Zero

This part explains the three technologies the repo is built on - PostgreSQL, Redis, and the idea of a message queue - assuming no prior exposure. If you know these, skip to Part 3.

2.1 PostgreSQL: the relational database

A relational database stores data in tables: named collections of rows, where every row has the same typed columns - much like a spreadsheet with an enforced schema. PostgreSQL ('Postgres') is one of the most widely used open-source relational databases. You talk to it in SQL, a declarative language: 'INSERT a row', 'SELECT rows WHERE status = pending', 'UPDATE this row SET status = processing'.

FOUNDRY MIRROR

A Postgres table is like a Foundry dataset with a strict schema - but designed for millions of tiny, instant reads and writes of individual rows, instead of bulk transformations of whole files. Where a Foundry pipeline rewrites a dataset in one big transaction per build, Postgres handles thousands of row-level changes per second, each one visible immediately.

Three Postgres concepts matter enormously for this repo:

- **TRANSACTIONS:** a group of changes that either ALL happen or NONE happen ('atomicity'). If the process dies halfway through a transaction, the database behaves as if the transaction never started. You end a transaction with COMMIT (make it permanent) or ROLLBACK (undo it).
- **ATOMIC CONDITIONAL UPDATES:** a statement like UPDATE jobs SET status='processing' WHERE id=X AND status='pending' checks the condition and applies the change as ONE indivisible operation. Two processes racing to run it cannot both succeed: the database serializes them, one matches the WHERE clause and updates the row, the other finds no matching row and changes nothing. This single property is the foundation of the whole repo's correctness - remember it.
- **INDEXES:** sorted lookup structures on chosen columns, so 'find all rows with status=pending' scans a small index instead of the whole table. The jobs table indexes status, priority, and (status, created_at).

FOUNDRY MIRROR

Foundry dataset transactions give you the same all-or-nothing idea at file scale: an aborted build never leaves a half-written dataset view. Postgres gives the same guarantee at the scale of individual rows, thousands of times per second.

2.2 SQLAlchemy and Alembic

The Python code never writes raw SQL strings by hand. SQLAlchemy is a library that maps Python classes to tables - the Job class in app/models/job.py IS the jobs table definition - and generates safe, parameterized SQL. Alembic is its companion migration tool: every schema change is a versioned script, and 'alembic upgrade head' replays them to bring any empty database up to the current schema. In docker-compose, a one-shot 'migrate' container runs this before anything else starts.

FOUNDRY MIRROR

Alembic migrations are to the database what committed transform code is to a Foundry pipeline: the schema is defined in version-controlled code and applied reproducibly, never clicked together by hand.

2.3 Redis: the in-memory data server

Redis is a server that keeps data structures in RAM and exposes commands to manipulate them over the network. Because everything lives in memory, operations take microseconds. It is not a general database - think of it as a very fast, shared scratchpad that many processes on many machines can read and write simultaneously. Redis offers several structure types; this repo uses two:

- **STREAM**: an append-only log of messages, each with an auto-assigned, ever-increasing ID. Producers append with `XADD`; consumers read entries. This repo uses three streams as its job queues (one per priority).
- **SORTED SET (ZSET)**: a set of string members, each with a numeric score, kept permanently sorted by score. Ask 'give me all members with score $\leq$ now' and you get them instantly. This repo stores future-scheduled job IDs in a sorted set, scored by their due timestamp.

One more Redis property matters here: single commands are atomic (the server executes one command at a time), and this repo's compose file turns on append-only-file persistence so Redis survives restarts. Even so, the design treats Redis as expendable: Postgres is always the source of truth, and Part 6 shows how the system rebuilds Redis state from Postgres if it is lost.

2.4 What a message queue / broker is (and where RabbitMQ fits)

A message queue is a buffer between a producer ('this job needs doing') and consumers ('I do jobs'). The producer appends a message and moves on; consumers pull messages when they have capacity. This decouples the two sides in time (producer does not wait), in space (different machines), and in rate (a burst of 10,000 submissions just makes the queue longer; workers drain it at their own pace).

RabbitMQ, Kafka, and AWS SQS are dedicated 'broker' products whose whole job is this. This repo instead builds a queue out of Redis Streams - a common, production-legitimate choice when you already run Redis and your needs are modest. The concepts are identical; only the product differs.

The hard part of any queue is delivery semantics. If a consumer takes a message and then crashes, is the message lost? Three contracts exist:

- **At-most-once**: message is deleted the moment it is handed out. A crash loses it. Fast but lossy.
- **At-least-once**: message stays 'checked out' until the consumer explicitly `ACKNOWLEDGES` (acks) it finished. A crash means the message is redelivered to someone else later. Nothing is lost, but the same message may be processed twice - so the processing must tolerate duplicates.
- **Exactly-once**: the marketing term. In real distributed systems it is achieved by at-least-once delivery PLUS deduplication on the consumer side - which is precisely what this repo does (Part 7.1).

FOUNDRY MIRROR

Foundry's build queue makes the same promise: if the module running your build is preempted or dies, the build does not evaporate - it goes back to the queue and runs again. You have seen builds restart from scratch after an infra hiccup; that is at-least-once delivery, experienced from the user side.

2.5 Concurrency and distributed systems: the two facts that generate every problem

This repo runs many independent processes: one API (which itself handles many requests at once), *N* workers, one ticker. They share **NO** memory - they can only coordinate through Postgres and Redis. Two facts follow, and every hard problem in Part 7 is a consequence of one or both:

- **FACT 1 - Interleaving**: between any two lines of your code, another process may run any number of its own lines. Code that reads a value, thinks about it, then writes ('check-then-act') is broken by default: the value may have changed between the read and the write. The fix is to make the check and the act a single atomic operation - which is exactly what the atomic conditional `UPDATE` from 2.1 provides.
- **FACT 2 - Partial failure**: any process can die between **ANY** two lines, and the others cannot directly tell. A worker that went silent might be dead, or might be slow, or the network might be dropping its packets - all three look identical from the outside. Systems handle this with timeouts ('silent too long = presumed dead') and by making recovery actions safe to apply even if the presumption was wrong.

A concrete race condition, because this trips everyone the first time. Two workers both want job J. Naive code: (1) read J's status - both see 'pending'; (2) both conclude 'it is free'; (3) both set status='processing' and both

run the job. The email gets sent twice. The read-then-write gap is the bug. The repo's fix: skip the read entirely and issue `UPDATE ... SET status='processing' WHERE id=J AND status='pending'` - the database guarantees exactly one worker's statement matches the row, and each worker learns from the row-count (1 or 0) whether it won.

FOUNDRY MIRROR

Foundry shields you from this constantly: two simultaneous builds writing the same dataset are serialized by dataset transactions, and you never see torn output. This repo is what it looks like to be the person BUILDING that shield rather than standing behind it.

3. The System at a Glance

`docker-compose.yml` defines six containers. Two are infrastructure (postgres, redis), one is a one-shot setup step (migrate), and three run this repo's code (api, worker, ticker) - all three built from the same image, differing only in the command they run.

- postgres - the database. Owns all job state. The ONLY place a job's status is authoritative.
- redis - holds the three queue streams, the delayed-jobs sorted set, and per-message delivery bookkeeping. Dispatch signals only - never the truth.
- migrate - runs 'alembic upgrade head' to create/upgrade the schema, then exits. Everything else waits for it.
- api - FastAPI HTTP server (`app/main.py`, routes in `app/api/routes.py`). Accepts submissions, answers status queries, handles cancel/retry, serves `/health` and `/stats`.
- worker - the job executor (`app/worker/runner.py`). An infinite loop: pull one job ID from the queue, claim it, run its handler, record the outcome, acknowledge. Scale it with: `docker compose up --scale worker=N`.
- ticker - the clock of the system (`app/ticker/runner.py`), exactly ONE replica. Every second it promotes scheduled jobs that are now due; periodically it also rescues jobs whose worker died, and repairs jobs that never made it into Redis.

FOUNDRY MIRROR

The Foundry mapping, service by service: api = the build service endpoint you hit when you click Build; postgres = the build/job history database behind the builds page; redis streams = the build queue; worker = a Spark module executor that picks up queued builds; ticker = the schedules service plus the janitor that notices dead builds. 'docker compose up --scale worker=5' is adding executors to the compute cluster.

The one design rule that explains everything else

Postgres is the source of truth; Redis is only a fast messenger. Every state change is committed to Postgres FIRST; the Redis message just says 'job <uuid> may be ready, go look at the database'. Messages carry no payload - only the job ID. Consequence: Redis can lose messages, deliver them twice, or be wiped entirely, and no job state is ever wrong - at worst a job waits until a repair loop re-sends its message. Keep this rule in mind through everything that follows; it is why each mechanism is safe.

4. Life of a Job: The Happy Path, End to End

Follow one job from submission to completion, with the actual code. A client submits:

```
POST /jobs
{
  "type": "email",
  "payload": {"to": "user@example.com", "subject": "hi"},
  "priority": "high"
}
```

Step 1 - API validates and records (app/api/routes.py, submit_job)

FastAPI parses the body into a JobSubmission schema. `validate_payload()` then checks the payload against the Pydantic schema registered for THIS job type (app/schemas/payloads.py) - an email payload missing 'to' is rejected with HTTP 422 before anything is stored. Then `_create_and_handoff()` inserts a row into the jobs table with status='pending' and COMMITs. The job now durably exists. Nothing has been queued yet.

Step 2 - API enqueues the pointer

`enqueue()` (app/queue/producer.py) runs XADD on the stream matching the job's priority - here jobs:stream:high - with one field: the job's UUID. Then `mark_synced()` flips the row's `is_synced_to_redis` flag to true. That flag exists for a subtle reason covered in Part 7.6: if the API dies BETWEEN the commit and the XADD, the row is stranded with `synced=false`, and the ticker will notice and re-send it. The client has already received HTTP 202 with the job ID.

Step 3 - A worker pulls the message (app/queue/consumer.py, read_priority)

Every worker loops on `read_priority()`: probe jobs:stream:high, then :normal, then :low, non-blocking, taking the first message found - so high-priority work is always drained first. Only if all three are empty does it fall into a single blocking read across all three (`block_ms=5000`), which parks the worker cheaply until something arrives. The read uses XREADGROUP - the consumer-group variant explained in Part 6 - which guarantees each message is handed to exactly one worker.

Step 4 - The worker claims the job in Postgres (app/worker/runner.py, process_job)

```
if not repo.claim_job(session, job_id):
    log.info("job.skipped", reason="not_claimable")
    return Outcome(ack=True, recycle=False, label="skipped")
```

`claim_job()` is the atomic conditional UPDATE from Part 2: set status='processing', started_at=now() WHERE id=? AND status IN ('pending','scheduled'). If it reports 0 rows changed, someone else already claimed the job (or it was cancelled) - the worker acks the message and moves on without running anything. This is the deduplication layer that turns at-least-once delivery into at-most-once execution.

Step 5 - The handler runs, inside a timeout

`run_handler()` looks up the function registered for 'email' in app/jobs/registry.py and calls it - wrapped in `run_with_timeout()` (app/worker/timeout.py), which runs it on a fresh thread and gives it `job_handler_timeout_s` (45s) to finish. The happy path: the handler sleeps its simulated 1-3 seconds and returns `{"message_id": "..."}.`

Step 6 - Completion is committed, THEN the message is acked

```
won = repo.complete_job(session, job.id, result, ...) # guarded UPDATE + COMMIT
...
return Outcome(ack=True, ...) # caller then runs XACK
```

`complete_job()` is another guarded UPDATE (only a 'processing' row can become 'completed'), storing the result JSON and `completed_at`. Only after that commit does the worker XACK the stream message - telling Redis 'delivery complete, stop tracking it'. The order is deliberate and load-bearing: if the worker died between commit and ack, the message would eventually be redelivered, the next worker's `claim_job()` would find the row already terminal, and it would simply ack and skip. Harmless. The reverse order (ack, then crash before commit) would lose the result forever with no redelivery to save it. When in doubt: make the durable write first, the cleanup signal second.

Step 7 - The client polls

```
GET /jobs/{id}
-> {"status": "completed", "result": {"message_id": "..."},
    "attempts": 1, "created_at": ..., "started_at": ..., "completed_at": ...}
```

FOUNDRY MIRROR

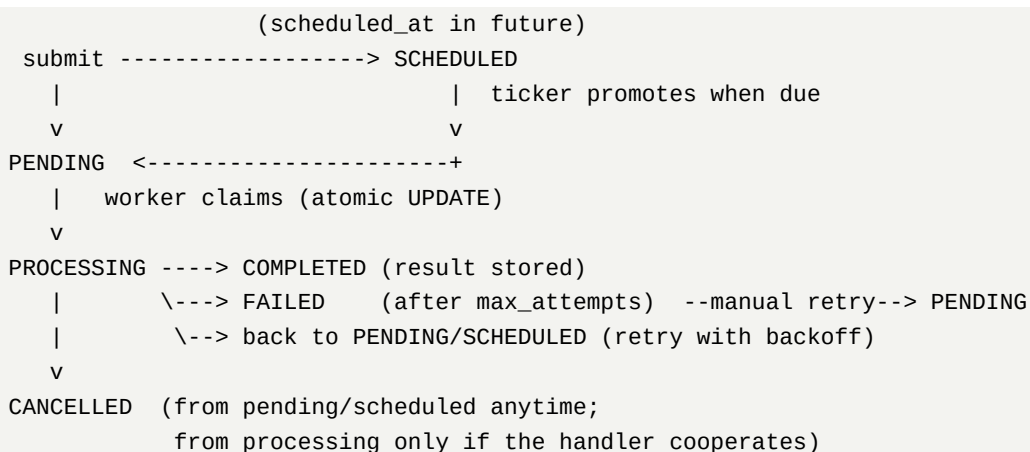
The whole arc mirrors a Foundry build exactly: submit (Build button) -> recorded (QUEUED on the builds page) -> picked up by an executor (RUNNING) -> result committed (SUCCEEDED, with artifacts and timing). Even the polling experience - refreshing the build page - is the same interaction this API's GET /jobs/{id} provides.

5. The Data Model: One Table That Tells the Whole Story

There is one core table, `jobs`, defined by the `Job` class in `app/models/job.py`. Reading it column by column is the fastest way to understand the system, because nearly every mechanism in Part 7 exists to correctly manage one or two of these columns.

- `id` (UUID, primary key) - random unique identifier, generated at insert. UUIDs (not 1,2,3...) so IDs are unguessable and can be generated without coordination.
- `type` - which handler runs this job: email, webhook, report, or batch.
- `payload` (JSONB) - the job's validated input, stored as queryable binary JSON.
- `status` - the heart of the model: scheduled, pending, processing, completed, failed, or cancelled. Indexed, because workers and the list endpoint constantly filter on it.
- `priority` - high, normal, or low. Decides WHICH Redis stream the job's message goes to.
- `result / error` (JSONB) - outcome of the final attempt: result on success, structured error info (exception type + message) on failure.
- `attempts / max_attempts` - retry bookkeeping: how many times this job has been tried, and the cap (default 4) before it is failed permanently.
- `created_at / started_at / completed_at` - lifecycle timestamps.
- `scheduled_at` - when a future-dated job should run; also reused as the 'retry after backoff' due time.
- `is_synced_to_redis` - repair flag: 'has this job's message actually been placed in Redis?' (Part 7.6).
- `progress` - percentage complete, used by the batch handler.
- `cancel_requested_at` - set by the cancel endpoint on an in-flight job; the batch handler polls for it (Part 7.7).
- `idempotency_key / idempotency_hash` - duplicate-submission protection (Part 7.5). A PARTIAL UNIQUE INDEX enforces key uniqueness at the database level - two simultaneous inserts with the same key physically cannot both succeed.

The status lifecycle



Every transition in this diagram is performed by a guarded `UPDATE` in `app/repository.py` - `claim_job`, `complete_job`, `fail_job`, `retry_to_pending`, `retry_to_scheduled`, `cancel_pending_or_scheduled`, `cancel_job`, `reset_failed_to_pending`. Each returns `True/False` for 'did I win the transition', and callers branch on it. There is no code path anywhere that sets a status without stating which prior status it expects - that discipline is what makes concurrent actors safe.

FOUNDRY MIRROR

The status enum maps one-to-one onto Foundry build states: `SCHEDULED` = a schedule that has not fired; `PENDING` = `QUEUED`; `PROCESSING` = `RUNNING`; `COMPLETED` = `SUCCEEDED`; `FAILED` = `FAILED`;

CANCELLED = CANCELED. The jobs table is the builds page as a table.

6. The Redis Side: Streams, Consumer Groups, and the Delayed Set

Everything in Redis is derived, disposable dispatch state. Five keys exist: three streams (`jobs:stream:high` / `:normal` / `:low`), one consumer group named 'workers' spanning them, and one sorted set (`jobs:delayed`).

6.1 Streams and XADD

A stream is an append-only log. `enqueue()` runs `XADD`, appending an entry whose only field is `job_id=<uuid>`. Entries get server-assigned, monotonically increasing IDs, so a stream is inherently ordered - within one priority stream, jobs are dispatched roughly first-in-first-out.

6.2 Consumer groups: how N workers share one stream without stepping on each other

If every worker simply read the stream, all of them would see every message. A CONSUMER GROUP fixes that: it is a named cursor over the stream, shared by all members. When a worker calls `XREADGROUP` asking for 1 new message, Redis advances the shared cursor and hands that message to THAT worker alone - no other group member will receive it. Each worker identifies itself with a unique consumer name (hostname + random suffix, built in `app/queue/consumer.py`).

Crucially, delivery is not deletion. The moment a message is handed out, Redis records it in the group's PENDING ENTRIES LIST (PEL): 'message 17 was delivered to worker_abc at 12:00:03 and is not yet acknowledged'. The message stays there until the worker calls `XACK`. The PEL is Redis's built-in answer to 'what was in flight when something died?' - and Part 7.2 shows the ticker mining it to rescue orphaned jobs. `XAUTOCLAIM` is the companion command: 'hand me every PEL entry idle longer than N ms', i.e. deliveries whose worker has gone silent.

FOUNDRY MIRROR

The PEL is Foundry's 'currently running builds' ledger. If an executor dies mid-build, Foundry does not forget the build existed - it knows exactly which builds were checked out and re-dispatches them. `XREADGROUP` = executor picks up a queued build; `XACK` = build reported terminal; `XAUTOCLAIM` = the janitor collecting builds whose executor stopped reporting.

6.3 Why three streams instead of one

Redis streams have no concept of priority - a stream is strictly append-ordered. Rather than fight that, the design makes priority a routing decision: three parallel streams, one per level, chosen once at enqueue time by `settings.stream_for_priority()`. Workers then impose priority at READ time by probing high -> normal -> low (Part 4, step 3). The alternative - one sorted set scored by priority - was rejected because a ZSET pop has no delivery tracking: no PEL, no ack, no idle-reclaim, so the entire crash-recovery story of 7.2 would have to be hand-built. DECISIONS.md section 3 records this trade-off.

6.4 The delayed sorted set

Streams cannot hold a message back until a future time. So future work lives elsewhere: `jobs:delayed`, a sorted set where each member is a job UUID and its score is the epoch timestamp when it becomes due. Two things put jobs there: submissions with a future `scheduled_at`, and retries waiting out their backoff delay. The ticker polls it every second: 'members with score <= now' -> promote each into its priority stream (Part 7.4). A sorted set makes that due-check a single cheap range query.

FOUNDRY MIRROR

`jobs:delayed` is the schedules service: 'this build should fire at 02:00' sits inert until the scheduler's clock-tick

promotes it into the build queue. Backoff-retries reuse the same mechanism, exactly like Foundry scheduling a failed build's automatic re-run a few minutes later.

7. The Hard Problems, One at a Time

This part is the heart of the repo - and of the assessment it was built for. Each section names a failure scenario that WILL happen in any distributed system, then walks through this codebase's answer. The recurring pattern: Redis provides speed and signaling, Postgres provides the guarded transition that makes correctness non-negotiable.

7.1 Duplicate pickup: two workers, one job

Threat: the same job runs twice - two emails sent, one report generated twice. Sources of duplication: stream redelivery after a crash (by design, at-least-once), the reaper re-enqueueing a job whose worker was merely slow, a human double-clicking retry.

Defense in two layers. Layer 1, Redis: within the consumer group, XREADGROUP hands each message to exactly one consumer - competing workers never even see the same message in the normal case. Layer 2, Postgres, the one that actually guarantees correctness: `claim_job()`'s atomic `UPDATE ... WHERE status IN ('pending','scheduled')`. However many duplicate messages exist for job J - two, five, it does not matter - every worker that receives one runs the same statement, and the database lets exactly ONE of them transition the row to 'processing'. Every loser gets `rowcount 0`, acks its redundant message, and walks away. Duplicate MESSAGES are tolerated; duplicate EXECUTION is impossible.

Notice what is NOT here: no `'if job.status == pending'` check in Python. That check-then-act shape is the race condition from Part 2.5. The condition lives inside the `UPDATE`'s `WHERE` clause, where the database evaluates it atomically with the write. Interviewers and code reviewers look for exactly this.

7.2 Worker crash recovery: the job stuck in PROCESSING

Threat: a worker claims a job, sets `status='processing'`, and then the container is OOM-killed. The worker cannot clean up - it is gone. Without recovery, the job sits in 'processing' forever and the client waits forever.

The design distinguishes two different kinds of 'stuck', handled at two layers:

- HUNG HANDLER, PROCESS ALIVE: every handler runs on a single-use thread with a 45-second budget (`run_with_timeout`, `app/worker/timeout.py`). On timeout the worker itself routes the job into the retry path immediately - no external help needed. But Python cannot forcibly kill a thread, so the abandoned thread may still be running. The worker therefore RECYCLES: it exits with code 1 after `max_handler_timeouts_before_recycle` (default: the very first timeout), and `docker-compose`'s `restart:on-failure` boots a fresh, clean replacement. A deliberately crude but airtight way to prevent zombie-thread resource leaks.
- DEAD PROCESS: nothing in-process can react, so recovery is external. The claimed message is still sitting in the PEL, unacked, its idle-time growing. The ticker's `reap_stale()` runs every 30s and calls `XAUTOCLAIM` with `min_idle = visibility_timeout_s` (60s): 'give me every delivery idle over a minute'. For each reclaimed job ID it checks Postgres and acts (see `_reap_one`, `app/ticker/runner.py`): status 'processing' -> the worker died mid-run, so route it through the standard retry path (7.3); status terminal -> ghost message, just ack it; status unsynced -> the crash happened before the Redis handoff, so finish that handoff inline. Every branch ends by acking the reclaimed message so the PEL stays clean.

A quiet invariant ties the layers together: the Settings validator (`app/core/config.py`) REFUSES to start if `job_handler_timeout_s >= visibility_timeout_s`. The in-process timeout (45s) always fires before the reaper's threshold (60s), so the reaper can never reclaim a job from a worker that is alive and still inside its handler budget - the reaper only ever sees the truly dead. Encoding an ordering assumption as a startup-time validation, rather than a comment, is a pattern worth stealing.

What about false positives - a live worker that somehow exceeds 60s of silence, gets reaped, then finishes

anyway? Even then, corruption is impossible: `complete_job()` is guarded ('only a processing row may complete'), so whichever side commits first wins the row, and the loser's write changes nothing. The worker logs 'job.complete_lost_to_reaper' and refuses to ack. The worst possible outcome is one extra execution of the handler - which is why job handlers in at-least-once systems should always be written to tolerate re-runs.

FOUNDRY MIRROR

Foundry equivalent: an executor is preempted mid-build; the build sits in RUNNING briefly; the platform notices the executor stopped heartbeating and requeues the build; you see a new attempt appear. Same threat, same shape of answer. One honest difference, recorded in DECISIONS.md section 5: Foundry uses live heartbeats ('the executor last reported 4s ago'), while this repo uses a fixed idle timeout - simpler, but detection is slower and one-size-fits-all. The author names heartbeats as the first thing they would add with more time.

7.3 Retries with backoff: failure is an input, not an exception

The webhook job fails 20% of the time by design, because transient failure is normal: external services blip, networks drop. Hammering a struggling service with instant retries makes things worse, so retries WAIT, and the waits grow - 'backoff'.

One function owns the entire policy: `schedule_retry_or_fail()` in `app/retry.py` - read it, it is 40 lines. Logic: let `n` = the attempt that just failed. If `n >= max_attempts (4)`: guarded UPDATE to 'failed', store the error JSON, done permanently. Otherwise look up the wait in `retry_backoff_schedule = [0, 30, 120]`: after the 1st failure, retry immediately (guarded UPDATE back to 'pending' + XADD right now); after the 2nd, 30 seconds; after the 3rd, 120 seconds - for the delayed cases the job goes to 'scheduled' and into the `jobs:delayed` sorted set, and the ticker promotes it when due. So a maximally unlucky job lives: try, fail, try, fail, wait 30s, try, fail, wait 120s, try, fail -> FAILED with the last error stored. A human can then inspect it and POST `/jobs/{id}/retry`, which resets it to pending with a fresh attempt budget.

The elegant part: WHO calls this function. A worker whose handler raised calls it synchronously - that is how 'first retry is immediate' is possible. A worker whose handler timed out calls it. And the ticker's reaper calls it for jobs whose worker died without getting to call anything. Handler exception, hang, and process death all converge on one code path, so attempt-counting and terminal-failure semantics cannot drift apart between failure modes. When you find yourself building a second, slightly different recovery path - stop and reroute it through the first.

FOUNDRY MIRROR

Foundry builds retry transient infra failures automatically with growing delays, then give up and mark the build FAILED for a human. The manual retry endpoint is the 'Rerun' button on a failed build.

7.4 Scheduled jobs and the ticker's clock-tick

Submit with `scheduled_at` in the future and the API stores the row as 'scheduled' and adds the ID to `jobs:delayed` scored by due-time - it never touches a stream. Every second, the ticker's `promote_due()` asks the sorted set for due members. For each batch it: looks up priorities in Postgres (skipping IDs whose row was cancelled or deleted - they are dropped, not enqueued), XADDs each survivor to its priority stream and ZREMS it from the set in one pipelined round-trip (`app/queue/delayed.py`), then flips the rows pending in one bulk UPDATE.

One performance subtlety: batches are capped at `ticker_batch_size (100)`, but if a tick promotes a FULL batch, the loop skips its 1-second sleep and drains again immediately. Without that, 10,000 jobs all due at 09:00 would trickle out over 100 seconds of artificial lag. With it, the ticker chews through the backlog as fast as Redis and Postgres allow. Look for this 'drain until not full' shape whenever a fixed-rate poller meets bursty input.

FOUNDRY MIRROR

This is Foundry's scheduler firing a time-based schedule: due schedules become queued builds. The burst-drain concern is the '2am nightly wave' - hundreds of schedules firing in the same minute must not serialize behind an

artificial once-per-tick cap.

7.5 Idempotency: the same submission, twice

Threat: a client submits, the response is lost to a network blip, the client (or its retry library) submits again. Without protection: two identical jobs, two emails. The client cannot distinguish 'my request was lost' from 'my response was lost' - only the SERVER can deduplicate.

The contract: the client sends an `idempotency_key` - any string of its choosing, e.g. 'welcome-email-user-4711'. In `submit_job` (`app/api/routes.py`): compute a canonical hash of `type+payload` (`app/idempotency.py` - 'canonical' means the JSON is serialized with sorted keys first, so key order cannot change the hash); if a job with this key already exists AND the hash matches, return that existing job with HTTP 200 - the duplicate creates nothing; if the key exists with a DIFFERENT hash, HTTP 409 - the client is reusing a key for different work, always a client bug worth loud rejection.

The race: two identical submissions arrive SIMULTANEOUSLY, both look up the key, both find nothing, both insert. Application code cannot close this gap - only the database can. The jobs table carries a partial unique index on `idempotency_key` (Part 5), so one INSERT succeeds and the other throws `IntegrityError`; the route catches it, rolls back, re-reads the row the winner created, and returns it. Both clients get the same job ID. The general lesson repeats 7.1: uniqueness under concurrency is enforced by constraints in the database, never by look-before-you-insert in application code.

FOUNDRY MIRROR

Foundry-adjacent idea: re-running a build on unchanged inputs does not duplicate output data - dataset transactions make the operation converge. The 409-on-hash-mismatch resembles Foundry refusing conflicting semantics on the same identity rather than guessing.

7.6 The `is_synced_to_redis` flag: repairing the seam between two systems

The API writes to two systems in sequence: Postgres COMMIT, then Redis XADD. Part 2.5's Fact 2 says the process can die exactly between them - leaving a committed, real job that no worker will ever hear about, because its message was never sent. This 'dual write' seam cannot be closed with a transaction, because no transaction spans Postgres and Redis. So instead of preventing the gap, the design DETECTS and REPAIRS it: every row starts with `is_synced_to_redis=false` and is flipped true only AFTER its XADD succeeds. The ticker's `reconcile_orphans()` runs every 60s: any non-terminal row still unsynced after a 10-second grace period gets its message re-sent (stream or delayed set, whichever fits its status) and is marked synced. Worst case, a job whose enqueue was interrupted starts about a minute late. This is 'self-healing over prevention', and it is also the mechanism that makes Redis fully expendable: wipe Redis, and reconciliation rebuilds the queue from Postgres.

7.7 Cancellation: easy until the job is running

Cancelling a pending or scheduled job is a guarded UPDATE like any other - if it wins, the row is 'cancelled', and any stale message for it dies later at the claim guard. (The ID is also ZREMed from the delayed set - a harmless no-op if absent.) Cancelling a PROCESSING job is fundamentally different: the handler is executing on a worker somewhere, and you cannot safely kill a Python thread from outside. So in-flight cancellation is COOPERATIVE: the cancel endpoint merely sets `cancel_requested_at` and returns HTTP 202 - 'requested, not guaranteed'. A handler must voluntarily poll for that flag and abandon ship; only the batch handler does (between items, via its context object), raising `JobCancelled`, which the worker records as a cancelled outcome with a partial summary. Email/webhook/report jobs are seconds long, so polling them was not worth it - a documented trade-off (DECISIONS.md section 4), and a realistic one: cooperative cancellation is how Spark, Kubernetes graceful shutdown, and most job systems actually behave.

The cancel route itself (`app/api/routes.py`) is a small concurrency lesson: the job may legally flap between

pending and processing (a retry landed) WHILE the cancel request is being handled, so the route loops up to 3 times over its guarded transitions - try cancel-as-queued, else try request-cancel-as-processing, else re-read and re-decide - rather than assuming the status it read a microsecond ago still holds. Statuses are moving targets; code that reads one and acts on it a line later is already stale.

FOUNDRY MIRROR

Killing a RUNNING Foundry build is the same experience: the build does not vanish the instant you click cancel - the executor notices the cancellation signal at its next check and winds down, sometimes finishing the current stage first. And a batch job's progress percentage is the build progress bar.

7.8 Graceful shutdown

When docker stops a container it sends SIGTERM, waits a grace period (50s for workers - deliberately longer than the 45s handler budget), then kills. The worker's signal handler (run_forever) just sets a flag; the loop finishes its CURRENT job completely - commit and ack included - and only then checks the flag and exits. No work is abandoned by a routine deploy or scale-down. And if the kill lands anyway? The job is recovered by 7.2, because crash recovery does not care WHY the worker died. That is the quiet payoff of good crash recovery: graceful shutdown becomes an optimization, not a correctness requirement.

8. Guided Tour of the Code

A map of app/, ordered roughly by how you should read it. Suggested reading order for a first pass: models/job.py -> repository.py -> api/routes.py -> queue/ -> worker/runner.py -> retry.py -> ticker/runner.py. That order follows the data instead of the directory listing.

- app/main.py - FastAPI startup: config, logging, DB/Redis clients, mounts routes.
- app/api/routes.py - all six endpoints: POST /jobs, GET /jobs/{id}, GET /jobs (filtered list with cursor pagination), POST /jobs/{id}/cancel, POST /jobs/{id}/retry, GET /health, GET /stats.
- app/api/deps.py - dependency injection: per-request DB session and Redis client.
- app/models/job.py - the Job table (Part 5). app/models/base.py - SQLAlchemy base.
- app/repository.py - THE most important file for correctness: every status transition as a guarded UPDATE returning won/lost. No other file changes a status.
- app/schemas/ - Pydantic layer: enums.py (status/type/priority), api.py (request/response shapes), payloads.py (per-job-type payload validation), results.py (result shapes).
- app/queue/producer.py - enqueue() = XADD. app/queue/consumer.py - consumer names, ensure_group(), ack(), and read_priority() with the high->normal->low probe. app/queue/delayed.py - the sorted-set helpers: schedule(), due_job_ids(), pipelined promote().
- app/worker/runner.py - the worker loop and process_job() (Part 4 / 7.2). app/worker/timeout.py - run_with_timeout thread wrapper. app/worker/context.py - PgJobContext: handler-facing helper for progress updates and cancellation polls. app/worker/__main__.py - 'python -m app.worker'.
- app/jobs/handlers.py - the four handlers. app/jobs/registry.py - explicit dict: type -> handler (no dynamic dispatch from user input, on purpose).
- app/retry.py - backoff_delay() + schedule_retry_or_fail() (Part 7.3).
- app/ticker/runner.py - promote_due(), reconcile_orphans(), reap_stale(), and the tick loop (7.2/7.4/7.6).
- app/idempotency.py - canonical_hash(). app/cursor.py - opaque pagination cursor encode/decode.
- app/core/config.py - Settings from env vars, incl. the timeout invariant. app/core/db.py / redis.py - client factories. app/core/logging.py - structlog JSON setup.
- app/observability.py - /health checks and /stats gathering (queue depths, per-status counts).

Conventions worth noticing

- Every log line is structured JSON via structlog, with job_id / message_id / consumer bound as context - one job's whole life across three services is a single filter query. Print statements are banned by project rule.
- Guarded transitions return booleans, and callers ALWAYS branch on 'won' - grep for 'won =' to find every concurrency decision point in the codebase.
- Configuration is env-vars via pydantic-settings; the containers receive DATABASE_URL and REDIS_URL in docker-compose.yml and nothing is hard-coded.

9. Testing, Running, and Observing It

9.1 Running the system

```
docker compose up --build          # api :8000, 1 worker, ticker, pg, redis
docker compose up --scale worker=4 # more executors

curl -X POST localhost:8000/jobs -H 'Content-Type: application/json' \
  -d '{"type": "webhook", "payload": {"url": "https://x.test/hook"}}'
curl localhost:8000/jobs/<id>      # poll status
curl localhost:8000/health        # dependency checks
curl localhost:8000/stats         # queue depths + per-status counts
```

Startup ordering is encoded in compose: postgres/redis must be healthy -> migrate runs the schema -> api/worker/ticker start. The webhook type is the fun one to demo: submit ten and watch ~two of them retry with backoff in the worker logs (docker compose logs -f worker).

9.2 The test suite

Run everything with: `uv run pytest`. (uv is the Python package/environment manager used here - the project rule is `uv run <cmd>`, never raw `pip/venv`.) Tests split into two tiers, and the split itself is a lesson:

- tests/unit/ - pure logic, no services needed: backoff arithmetic, timeout wrapper, payload validation, handlers (with randomness injected so the webhook's '20% failure' is forced deterministically), config invariants, cursor encoding, idempotency hashing.
- tests/integration/ - run against REAL Postgres and Redis from compose: pickup through the consumer group, the full retry flow, the reaper reclaiming an unacked message, delayed promotion, cancellation, idempotent submission through the API, batch progress, worker end-to-end, migrations applying cleanly.

Why real Redis instead of a fake? The repo's git history contains the answer: a `fakereis` dependency was deliberately REMOVED. The system's guarantees live in exact Redis semantics - PEL bookkeeping, XAUTOCLAIM idle rules, group cursors. A fake that approximates those semantics passes tests precisely where the real thing would break, which is worse than no test. Note also HOW crash recovery is tested (tests/integration/test_reaper.py): no process is killed - the test reads a message, deliberately never acks, then invokes the reaper with a tiny idle threshold. Producing the exact intermediate STATE a crash leaves behind is faster and more deterministic than staging the crash.

FOUNDRY MIRROR

Analogy: unit tests = checks on transform logic with mocked inputs; integration tests = actually running the pipeline against real infrastructure. And 'don't trust a fake of your platform' is why Foundry pipeline validation ultimately happens on Foundry, not a local Spark that merely resembles it.

9.3 Observing it

GET /health answers 'can I reach Postgres and Redis' (503 if not - suitable for container healthchecks; compose wires it in). GET /stats reports per-status job counts plus Redis queue depths - the numbers that matter operationally: pending counts rising with idle workers means dispatch is broken; delayed-set size rising past due times means the ticker is down (it is single-instance - the system's one 'watch this' component); PEL size growing means workers are dying or stuck. Every log line ships as JSON with the job ID attached, so 'grep the job_id' reconstructs any job's full biography across api, worker, and ticker.

10. Limits, Trade-offs, and a Glossary

10.1 Honest limits (all documented in DECISIONS.md)

- Strict priority can starve: a continuous flood of high-priority jobs delays normal/low indefinitely. Documented escape hatch: dedicate a worker to the low stream via its stream list.
- Recovery is timeout-based, not heartbeat-based: a dead worker's job waits up to ~60s before rescue, and the threshold must be sized for the slowest job type. Heartbeats are the named 'with more time' improvement.
- The ticker is a single instance: its loops are not safe to run twice concurrently (no leader election), so it must not be scaled out. Its workload is tiny; its liveness deserves the closest monitoring.
- One retry policy for all job types: max_attempts and the backoff table are global settings, not per-job.
- In-flight cancellation only works for batch jobs (cooperative polling, 7.7).
- No authentication: out of scope for the assessment; the first thing to add before real exposure.

10.2 Glossary

- Job - one unit of background work: a type, a payload, a lifecycle.
- Queue - buffer between 'work requested' and 'work done'; here, Redis streams.
- Producer / Consumer - the side that enqueues / the side that pulls and executes.
- Broker - dedicated queue middleware (RabbitMQ, Kafka, SQS); Redis plays that role here.
- Stream - Redis append-only log; XADD appends, XREADGROUP reads via a group.
- Consumer group - named shared cursor so N consumers split one stream without overlap.
- PEL (Pending Entries List) - Redis's ledger of delivered-but-unacked messages; the crash-recovery breadcrumb trail.
- XACK - consumer's 'done, stop tracking this delivery'. XAUTOCLAIM - 'give me deliveries idle longer than N ms' (the reaper's tool).
- Sorted set (ZSET) - Redis structure sorted by numeric score; here, job IDs scored by due-time.
- Transaction - all-or-nothing group of database changes.
- Atomic operation - indivisible; no other process can interleave inside it.
- Guarded transition - UPDATE with the expected prior state in its WHERE clause; returns won/lost. This repo's signature move.
- Race condition - bug from unlucky interleaving of concurrent actors, e.g. check-then-act.
- Idempotent - safe to apply twice with the same effect as once.
- At-least-once - delivery contract: never lost, possibly duplicated; pair with dedup to get effective exactly-once.
- Visibility timeout - idle time after which an unacked delivery is presumed abandoned (60s here).
- Backoff - growing delays between retries ([0s, 30s, 120s] here).
- Reaper / Reconciler / Promoter - the ticker's three loops: rescue dead workers' jobs / repair never-enqueued jobs / release due scheduled jobs.
- Worker recycling - worker exits after a handler timeout so the orchestrator restarts it clean.
- Ticker - this repo's scheduler-plus-janitor service; single instance.

10.3 Where to go from here

Read DECISIONS.md next - now that you know the mechanisms, the trade-off discussions will land. Then do the exercise that teaches the most: bring the stack up, submit a batch job, 'docker compose kill worker' mid-run,

and watch the logs as the ticker reaps it and a fresh worker finishes the retry. Recovery you have personally watched is recovery you understand. After that, pick any limit from 10.1 and sketch the fix - per-job retry policies is the gentlest on-ramp; heartbeats is the most instructive.